

EX.NO.12	NODE.JS AND MONGODB
DATE: 28.09.26	

AIM:

To develop a Node.js and Express server connected to MongoDB that performs create, read, update and delete (CRUD) operations on library member records.

ALGORITHM:

1. Create a project folder and run `npm init -y`.
2. Install the packages `express` and `mongodb` with `npm install`.
3. Start the MongoDB server and open MongoDB Compass at `mongodb://127.0.0.1:27017`.
4. Create the database `library` and the collection `members`.
5. Write `server.js` that connects to MongoDB and enables JSON parsing in Express.
6. Create the routes `POST /members`, `GET /members`, `PUT /members/:id` and `DELETE /members/:id`.
7. Run `node server.js` and test every route from the browser page and Compass.

CODE:**server.js:**

```
const express = require("express");
const { MongoClient } = require("mongodb");

const app = express();
app.use(express.json());
app.use(express.static("public"));

const client = new MongoClient("mongodb://127.0.0.1:27017");
let members;

async function start() {
  await client.connect();
  members = client.db("library").collection("members");
  console.log("MongoDB connected - library.members");

  // CREATE
  app.post("/members", async (req, res) => {
    await members.insertOne(req.body);
    res.send("Member added successfully");
  });

  // READ
  app.get("/members", async (req, res) => {
    res.json(await members.find().toArray());
  });

  // UPDATE
  app.put("/members/:id", async (req, res) => {
    await members.updateOne({ memberId: req.params.id }, { $set: req.body });
    res.send("Member updated successfully");
  });

  // DELETE
  app.delete("/members/:id", async (req, res) => {
    await members.deleteOne({ memberId: req.params.id });
    res.send("Member deleted successfully");
  });
}
```

```
app.listen(4000, () => console.log("Server running at http://localhost:4000"));
}
start();
```

Sample document:

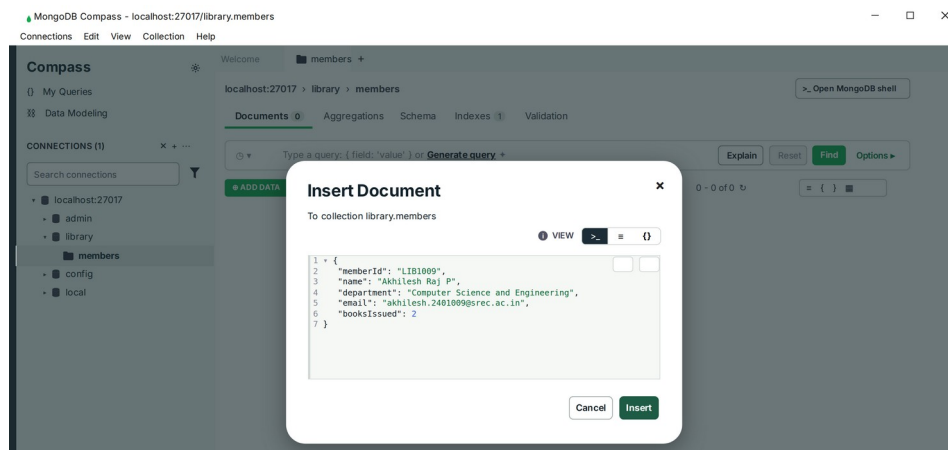
```
{
  "memberId": "LIB1009",
  "name": "Akhilesh Raj P",
  "department": "Computer Science and Engineering",
  "email": "akhilesh.2401009@srec.ac.in",
  "booksIssued": 2
}
```

OUTPUT:

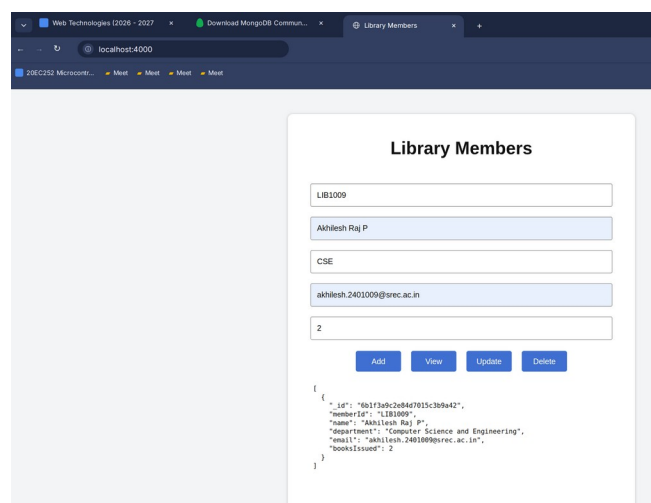
Server start

```
D:\WT_LAB\Programs\Exp12>node server.js
MongoDB connected - library.members
Server running at http://localhost:4000
```

ADD - insert document in MongoDB Compass



VIEW



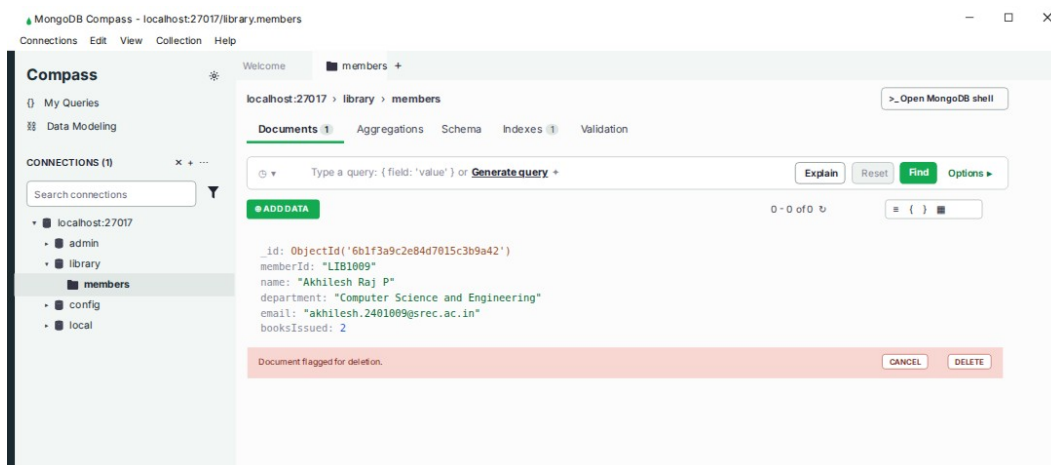
UPDATE

Library Members

AddViewUpdateDelete

Member updated successfully

DELETE - document flagged for deletion in Compass



RESULT:

Thus, a Node.js and MongoDB library member management system was developed and the CRUD operations were verified successfully.